The Evolution of Sonic Assistant

A technical and personal history of how a simple local RAG experiment became a cloud-deployed, multi-agent AI system with memory, tools and safeguards. Updated October 2026.

1. It started as a local RAG experiment (June 2026)

Sonic Assistant began as a small personal challenge: could I build my own retrieval-augmented generation system from scratch? No cloud, no microservices, no orchestration. Just a local Python script, a vector store, and a simple retrieval pipeline. I wanted to understand how embeddings work, how chunking affects retrieval, how hybrid search improves accuracy, and how to build a conversational loop around a RAG model. It wasn't fancy, but it was the first time I built an AI system end to end by myself. The first commit is dated June 14, 2026, and it lit the spark.

2. The first major leap: PAAPP (July 2026)

Once the local RAG model worked, I wanted Sonic Assistant to feel like a real application and not just a script. That meant authentication, user identity, persistent storage, multi-user support, real API endpoints and a headless backend service. So I built PAAPP, a standalone FastAPI microservice for time tracking, event logging and user-specific data. It was my first microservice and my first distributed architecture. Later, as Sonic gained real integrations (Google Calendar, Gmail, GitHub), the built-in productivity modules from this period (notes, a task board, a calendar module and an insights dashboard) were retired in October in favour of connecting to the tools people already use.

3. The second leap: LangGraph and NetworkX (July 2026)

Once the service layer existed, I wanted Sonic to reason across documents and not just retrieve them. On July 1 I replaced the original LangChain pipeline with LangGraph, which gave me multi-node workflows, branching logic, retrieval grading, query rewriting, fallback paths and stateful reasoning. I added NetworkX as a separate piece: an entity-relationship graph that is searched during retrieval and adds how-things-relate lines to the context, alongside lexical and hybrid (vector plus keyword) search. LangGraph is the orchestrator; NetworkX is the graph of what the documents say about each other.

4. The third leap: a real multi-agent design (July 2026)

On July 8 the full multi-agent workflow shipped as v0.8: a reasoner that reads each message, a coordinator that plans which nodes run, a retriever with relevance grading and query rewriting, a summarizer, a formatter, and memory nodes. These communicate through shared state, conditional edges, fallback nodes and retry logic. This is where Sonic became modular and extensible.

5. Cloud deployment and public access (July to August 2026)

The hardest step was turning it into a real, public application: the React frontend on Vercel, the FastAPI backend on Render, Clerk authentication, CORS, a custom domain (sonicassistant.com), and a lot of production debugging: stale bundles, malformed URLs, routing between services. In August it was embedded in a partner fitness website as a chat widget, which gave it real commercial use, and I connected it to errAgent, a separate incident-monitoring service, so production errors are analysed and fixed through pull requests.

6. It started to remember (September 2026)

In September Sonic stopped treating every conversation as the first. I built long-term memory: short facts about each person that are reinforced when repeated, replaced when they change, and decay when they stop being true; embedded notes from earlier conversations; and passive recall that brings in only what is genuinely relevant. Later came patterns across conversations, links between related facts, a Memory page where people can read, edit and delete everything stored, and a rule that Sonic must never pretend to remember something it doesn't.

7. It learned to use tools, with approval (September to October 2026)

Sonic gained a tool agent: it can read and search a GitHub repository, compare branches, summarise pull requests, run Python in a WebAssembly sandbox, read web pages, use a headless browser, read and draft with Google Calendar, Gmail, Drive and Docs, and work on a folder from the user's own computer, including creating and editing Word documents as new copies. Everything that changes the outside world is proposed first and happens only after the person approves it, and local files are only ever written by the user's own browser, with an undo.

8. It learned to be careful (October 2026)

October was about trust. I added a safety layer that notices when someone is struggling and responds with care, starting with the people in their life before any hotline; emotional attunement that carries across topic changes; a written profile and an honest changelog so Sonic only says true things about itself; spending limits per person and overall; log hygiene so private message content never reaches logs; and a growing set of automated tests (more than 3,500) and model evaluations, with CI that the agent can read. The interface was redesigned around the conversation, with a mascot, Patchy, who shows what Sonic is doing.

9. Where it stands today

Sonic Assistant is a multi-tenant, multi-agent AI application with retrieval over isolated knowledge bases, long-term memory, a tool agent with approval gates, a safety layer, and cost controls, running on Vercel, Render, MongoDB Atlas and Google Gemini. It has more than three hundred commits

since June 14, 2026, and I designed, directed and own every part of it: the architecture, the deployment and the debugging.

10. What this evolution says about me

Sonic Assistant's evolution mirrors my own: from help desk, to enterprise systems, to full-stack engineering, to AI systems, to cloud architecture. I didn't start as a software engineer. I wasn't handed a roadmap or given a team. I built this because I loved working on the Knowledge Assistant, because I wanted to challenge myself, and because I want a chance at a software engineering role. I built it because I know I can do the work, even if I've never had the title. Sonic Assistant is more than a project. It's my story, my growth, and my proof.